Java Utility Classes

Class / Feature	Purpose	Example Program	Output
Arrays	Perform operations on arrays	int[] a={30,10,20}; Arrays.sort(a); System.out.println(Arrays.toString(a));	[10, 20, 30]
Collections	Perform operations on collections	ArrayList<Integer> a=new ArrayList<>(Arrays.asList(30,10,20)); Collections.sort(a); System.out.println(a);	[10, 20, 30]
ArrayList	Store a dynamic list of elements	ArrayList<String> a=new ArrayList<>(); a.add("Java"); a.add("C"); System.out.println(a);	[Java, C]
LinkedList	Store elements using a linked list	LinkedList<String> a=new LinkedList<>(); a.add("A"); a.addFirst("Start"); System.out.println(a);	[Start, A]
HashSet	Store unique values	HashSet<Integer> s=new HashSet<>(); s.add(10); s.add(20); s.add(10); System.out.println(s);	[20, 10]*
TreeSet	Store unique values in sorted order	TreeSet<Integer> s=new TreeSet<>(); s.add(30); s.add(10); s.add(20); System.out.println(s);	[10, 20, 30]
HashMap	Store key-value pairs	HashMap<Integer,String> m=new HashMap<>(); m.put(101,"Ashok"); m.put(102,"Ravi"); System.out.println(m.get(101));	Ashok
TreeMap	Store key-value pairs sorted by key	TreeMap<Integer,String> m=new TreeMap<>(); m.put(103,"Kumar"); m.put(101,"Ashok"); System.out.println(m);	{101=Ashok, 103=Kumar}
Iterator	Traverse collection elements	ArrayList<String> a=new ArrayList<>(Arrays.asList("A","B")); Iterator<String> i=a.iterator(); while(i.hasNext()) System.out.println(i.next());	A B
Scanner	Read input from keyboard	Scanner sc=new Scanner(System.in); String name=sc.nextLine(); System.out.println("Hello "+name);	Hello Ashok
Random	Generate random values	Random r=new Random(); System.out.println(r.nextInt(10));	7
StringTokenizer	Split a string into tokens	StringTokenizer st=new StringTokenizer("Java is easy"); while(st.hasMoreTokens()) System.out.println(st.nextToken());	Java is easy
Date	Represent date and	Date d=new Date(); System.out.println(d);	Current date and time

	time		
Calendar	Access date/time components	Calendar c=Calendar.getInstance(); System.out.println(c.get(Calendar.YEAR));	2026

1. Arrays

Purpose: Perform operations on arrays.

```
import java.util.Arrays;

class Main {
   public static void main(String[] args) {
      int[] a = {30, 10, 20};

      Arrays.sort(a);

      System.out.println(Arrays.toString(a));
   }
}
```

[10, 20, 30]

2. Collections

Purpose: Perform operations on collections.

```
import java.util.*;

class Main {
   public static void main(String[] args) {
      ArrayList<Integer> a =
         new ArrayList<>(Arrays.asList(30, 10, 20));

      Collections.sort(a);

      System.out.println(a);
   }
}
```

[10, 20, 30]

3. ArrayList

Purpose: Store a dynamic list of elements.

```
import java.util.*;

class Main {
```

```
    public static void main(String[] args) {
        ArrayList<String> a = new ArrayList<>();

        a.add("Java");
        a.add("C");

        System.out.println(a);
    }
}
```
[Java, C]

4. LinkedList

Purpose: Store elements using a linked list.

```
import java.util.*;

class Main {
    public static void main(String[] args) {
        LinkedList<String> a = new LinkedList<>();

        a.add("A");
        a.addFirst("Start");

        System.out.println(a);
    }
}
```
[Start, A]

5. HashSet

Purpose: Store unique values.

```
import java.util.*;

class Main {
    public static void main(String[] args) {
        HashSet<Integer> s = new HashSet<>();

        s.add(10);
        s.add(20);
        s.add(10);

        System.out.println(s);
    }
}
```
[20, 10]
Note: HashSet does not guarantee order. The output may vary.

6. TreeSet

Purpose: Store unique values in sorted order.

```
import java.util.*;

class Main {
    public static void main(String[] args) {
        TreeSet<Integer> s = new TreeSet<>();

        s.add(30);
        s.add(10);
        s.add(20);

        System.out.println(s);
    }
}
```

[10, 20, 30]

7. HashMap

Purpose: Store key-value pairs.

```
import java.util.*;

class Main {
    public static void main(String[] args) {
        HashMap<Integer, String> m = new HashMap<>();

        m.put(101, "Ashok");
        m.put(102, "Ravi");

        System.out.println(m.get(101));
    }
}
```

Ashok

8. TreeMap

Purpose: Store key-value pairs sorted by key.

```
import java.util.*;

class Main {
    public static void main(String[] args) {
        TreeMap<Integer, String> m = new TreeMap<>();
```

```
        m.put(103, "Kumar");
        m.put(101, "Ashok");

        System.out.println(m);
    }
}
```
{101=Ashok, 103=Kumar}

9. Iterator

Purpose: Traverse collection elements.

```
import java.util.*;

class Main {
    public static void main(String[] args) {
        ArrayList<String> a =
            new ArrayList<>(Arrays.asList("A", "B"));

        Iterator<String> i = a.iterator();

        while (i.hasNext()) {
            System.out.println(i.next());
        }
    }
}
```
A
B

10. Scanner

Purpose: Read input from keyboard.

```
import java.util.*;

class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String name = sc.nextLine();

        System.out.println("Hello " + name);

        sc.close();
    }
}
```
Input

```
Ashok
Output
Hello Ashok
```

11. Random

Purpose: Generate random values.

```
import java.util.*;

class Main {
   public static void main(String[] args) {
      Random r = new Random();

      System.out.println(r.nextInt(10));
   }
}
7
```

Note: nextInt(10) generates a random integer from 0 to 9.

12. StringTokenizer

Purpose: Split a string into tokens.

```
import java.util.*;

class Main {
   public static void main(String[] args) {
      StringTokenizer st =
         new StringTokenizer("Java is easy");

      while (st.hasMoreTokens()) {
         System.out.println(st.nextToken());
      }
   }
}
Java
is
easy
```

13. Date

Purpose: Represent date and time.

```
import java.util.*;

class Main {
```

```
    public static void main(String[] args) {
        Date d = new Date();

        System.out.println(d);
    }
}
```
Thu Oct 29 09:22:00 IST 2026

14. Calendar

Purpose: Access date/time components.

```
import java.util.*;

class Main {
    public static void main(String[] args) {
        Calendar c = Calendar.getInstance();

        System.out.println(c.get(Calendar.YEAR));
    }
}
```
2026